

# PLANO DE INTEGRAÇÃO E ENTREGA CONTÍNUA

Cidade Virtual - PI - III

Avaliação do Módulo 4 - Integração e Entrega Contínua

Maio de 2026

|                      |                                                    |
|----------------------|----------------------------------------------------|
| Nome Completo:       | Diego Serafim de Sousa                             |
| Disciplina:          | Projeto Integrador de Tecnologia da Informação III |
| Semestre:            | 2026/1                                             |
| Curso:               | Tecnologia da Informação                           |
| Público-alvo:        | Moradores, comerciantes e autônomos.               |
| Local de realização: | Cabanagem, Belém, Pará                             |

## 1 TEMA DA AÇÃO

PLANO DE CI/CD PARA O PROJETO CIDADE VIRTUAL - INTEGRAÇÃO E ENTREGA CONTÍNUA

## 2 RESUMO

Este documento apresenta o Plano de Integração e Entrega Contínua (CI/CD) para o projeto Cidade Virtual, plataforma web desenvolvida para conectar moradores, comerciantes e autônomos do bairro e da cidade. O plano estabelece a automação de processos de linting, testes unitários e de integração, verificação de segurança, análise estática de código e deploy automatizado para ambientes de homologação e produção. São propostas ferramentas como GitHub Actions, Vercel, SonarCloud, Husky e CodeRabbit (IA). O cronograma de implantação é de 4 semanas, com objetivo de reduzir o tempo de deploy manual em 70%, aumentar a confiabilidade das entregas e proporcionar feedback imediato sobre a qualidade do código.

**Palavras-chave:** CI/CD. Integração contínua. Entrega contínua. Cidade Virtual. GitHub Actions.

## 3 INTRODUÇÃO

O projeto Cidade Virtual vem sendo desenvolvido desde 2022 pelo estudante Diego Serafim de Sousa, sendo continuamente aprimorado no âmbito das disciplinas do Projeto Integrador da UFMS. Trata-se de uma plataforma web desenvolvida na pilha MERN (MongoDB, Express.js, React.js, Node.js) que visa conectar moradores, comerciantes locais e autônomos da comunidade, promovendo o fortalecimento da economia local e a integração comunitária.

Atualmente, o ciclo de desenvolvimento do projeto é realizado de forma manual: os testes são executados localmente, o deploy na Vercel é feito através do push direto

ou pela interface web, e não tem verificações automatizadas de qualidade ou segurança antes da integração do código. Esse cenário traz riscos de defeitos não identificados, inconsistências de estilo e vulnerabilidades que podem impactar a comunidade usuária.

### 3.1 Objetivos do plano

O presente Plano de CI/CD tem como objetivo:

Tabela 1 - Objetivos e benefícios esperados do Plano de CI/CD

| ID  | Objetivo                                                                             | Benefício esperado                                          |
|-----|--------------------------------------------------------------------------------------|-------------------------------------------------------------|
| OG1 | Automatizar a execução de testes (unitários, integração) a cada push ou pull request | Detecção precoce de defeitos, redução de trabalho           |
| OG2 | Automatizar a verificação de conformidade de código (lint, formatação, tipos)        | Padronização do código, facilidade de manutenção            |
| OG3 | Automatizar a verificação de segurança (dependências vulneráveis)                    | Mitigação de riscos de segurança                            |
| OG4 | Automatizar o deploy em ambiente de homologação e produção                           | Entregas mais rápidas, conscientes e sem intervenção manual |
| OG5 | Integrar ferramentas de análise estática e revisão com IA                            | Melhoria contínua da qualidade e aprendizado                |

Fonte: Elaborado pelo autor (2026)

## 4 ESCOPO E TECNOLOGIAS

### 4.1 Escopo de aplicação

As práticas de CI/CD descritas neste plano serão aplicadas ao back-end (Node.js/Express) e ao front-end (React.js) do projeto Cidade Virtual abrangendo:

- Repositório GitHub do projeto
- Branch `main` (produção)
- Branch `develop` (homologação)
- Branches `feature/*` e `hotfix/*` (desenvolvimento)

Estão fora do escopo inicial: Automação de testes de carga, testes de segurança avançados (penetração manual) e deploy em múltiplas regiões.

## 4.2 Ferramentas e plataformas

Tabela 2 - Ferramentas, plataformas e justificativas do pipeline de desenvolvimento

| Ferramentas        | Finalidade                                          | Justificativa                                                                                |
|--------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------|
| GitHub             | Controle de versão e plataforma de CI/CD            | Amplamente adotado, integração nativa com GitHub Actions, gratuito para projetos open-source |
| GitHub Actions     | Automação de workflows de CI/CD                     | Flexível, baseado em YAML, vasta biblioteca de ações pré-prontas                             |
| ESLint, Prettier   | Conformidade de código (liting e formatação)        | Padrão da comunidade JavaScript/TypeScript                                                   |
| Jest, Supertest    | Testes unitários e de integração                    | Já utilizado no projeto, compatível com Node.js/React.js                                     |
| npm audit / Snyk   | Verificação de vulnerabilidades em dependências     | Identificação de CVEs conhecidas                                                             |
| SonarCloud         | Análise estática de qualidade e cobertura de testes | Métricas de dívida técnica, duplicação, complexidade                                         |
| Vercel             | Hospedagem e deploy contínuo                        | Já utilizado no projeto, integração nativa com GitHub                                        |
| CodeRabbit         | Revisão automatizada de código com IA               | Sugestões inteligentes em pull requests                                                      |
| Husky, lintstarged | Hooks de pré-commit                                 | Execução de verificações antes do commit local                                               |

Fonte: Elaborado pelo autor (2026)

## 4.3 Ambientes e seus propósitos

Tabela 3 - Ambientes de implantação, propósitos e níveis de acesso de software

| Ambiente                | Propósito                                        | Acesso                                                                            |
|-------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------|
| Desenvolvimento (local) | Implementação de funcionalidades, testes manuais | Máquina do desenvolvedor                                                          |
| Homologação (preview)   | Validação de pull requests, testes integrados    | URL gerada automaticamente pela Vercel para cada PR                               |
| Produção                | Disponibilização pública para a comunidade       | <a href="https://cidadevirtual.vercel.app/">https://cidadevirtual.vercel.app/</a> |

Fonte: Elaborado pelo autor (2026)

## 5 PRÁTICAS DE INTEGRAÇÃO CONTÍNUA (CI)

### 5.1 Visão geral

A integração Contínua será implementada por meio de workflows do GitHub Actions executados automaticamente nos seguintes eventos:

- `push` para as branches `develop` e `main`
- `pull_request` direcionado à branch `main`

### 5.2 Atividades automatizadas de CI

Tabela 4 - Atividades automatizadas, ferramentas e ações para implantação de CI

| Atividade                | Ferramenta                    | Justificativa / Benefício                                                                      | Atividades para implantação                                                                                                                 |
|--------------------------|-------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Linting e formatação     | ESLint, Prettier              | Garante padrão consistente de código; evita conflitos de estilo em revisões; reduz erro humano | Configurar <code>.eslintrc.js</code> , <code>.prettierrc</code> ; adicionar scripts <code>npm run lint</code> e <code>npm run format</code> |
| Verificação de tipos     | TypeScript (tsc)              | Captura erros de tipo antes da execução; melhora a documentação implícita                      | Configurar <code>tsconfig.json</code> ; adicionar script <code>npm run type-check</code>                                                    |
| Testes unitários         | Jest                          | Detecta regressões em unidades isoladas; cobertura mínima de 70%                               | Configurar <code>jest.config.js</code> ; criar testes para <code>services</code> e <code>utils</code>                                       |
| Testes de integração     | Supertest, Jest               | Valida a comunicação entre API e banco de dados (MongoDB em memória)                           | Criar testes para endpoints principais (cadastro, busca, avaliação)                                                                         |
| Verificação de segurança | <code>npm audit</code> / Snyk | Identifica dependências com vulnerabilidades conhecidas (CVEs)                                 | Adicionar <code>npm audit --production</code> ao workflow                                                                                   |
| Análise estática         | SonarCloud                    | Mede dívida técnica, duplicação de código, complexidade ciclomática                            | Integrar SonarCloud ao GitHub; adicionar token ao repositório                                                                               |

Fonte: Elaborado pelo autor (2026)

### 5.3 Workflow de CI proposto

name: CI - Validação de Qualidade

on:

push:

branches: [develop, main]

pull\_request:

branches: [main]

jobs:

validate:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- uses: actions/setup-node@v3

with:

node-version: '18'

cache: 'npm'

- run: npm ci

- run: npm run lint

- run: npm run type-check

- run: npm run test:unit -- --coverage

- run: npm run test:integration

- run: npm audit --production

- name: SonarCloud Scan

uses: SonarSource/sonarcloud-github-action@master

env: SONAR\_TOKEN: \${ secrets.SONAR\_TOKEN }

### 5.4 Atividades para implantação da CI

Tabela 5 - Etapas e estimativa de prazos para implantação da infraestrutura de CI

| Etapa | Descrição                                                  | Prazo estimado |
|-------|------------------------------------------------------------|----------------|
| 1     | Configurar ESLint, Prettier e TypeScript no projeto        | 2h             |
| 2     | Implementar testes unitários para serviços críticos        | 8h             |
| 3     | Implementar testes de integração para endpoints principais | 6h             |
| 4     | Configurar SonarCloud e integrar ao repositório            | 2h             |
| 5     | Criar workflow YAML no GitHub Actions                      | 2h             |
| 6     | Ajustar variáveis de ambientes e secrets                   | 1h             |

Fonte: Elaborado pelo autor (2026)

## 6 PRÁTICAS DE ENTREGA CONTÍNUA (CD)

### 6.1 Visão geral

A entrega contínua será implantada por meio da integração nativa da Vercel com o GitHub, complementada por workflows do GitHub Actions para validações adicionais antes do deploy em produção.

### 6.2 Atividades automatizadas de CD

Tabela 6 - Atividades automatizadas, ferramentas e ações para implantação de CD

| Atividade                        | Ferramenta                    | Justificativa / Benefício                                                                       | Atividades para implantação                                         |
|----------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Deploy automático em homologação | Vercel (Preview Deployments)  | Cada pull request gera uma URL de preview única; permite testar funcionalidades antes do merger | Conectar repositório à Vercel; configurar integração automática     |
| Deploy em produção               | Vercel, GitHub Actions        | Ao fazer merger na main (após CI passar), fazer deploy automático                               | Configurar action amondnet/vercel-action ou utilizar webhook nativo |
| Versionamento semântico          | GitHub Tags, Release          | Criar tags v1.2.3 e releases automáticas a partir de merges na main                             | Configurar workflow que cria tag baseada no commit                  |
| Notificações de deploy           | GitHub Actions, Discord/Email | Alertar sobre sucesso/falha no deploy                                                           | Adicionar step de notificação (discord webhook)                     |

Fonte: Elaborado pelo autor (2026)

### 6.3 Workflow de CD proposto

name: CD - Deploy para Vercel

on:

push:

branches: [main]

workflow\_run:

workflows: ["CI - Validação de Qualidade"]

types:

- completed

jobs:

deploy-production:

if: \${{ github.event.workflow\_run.conclusion == 'success' }}

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- name: Deploy to Vercel  
uses: amondnet/vercel-action@v20  
with:
  - vercel-token: \${{ secrets.VERCEL\_TOKEN }}
  - vercel-org-id: \${{ secrets.ORG\_ID }}
  - vercel-project-id: \${{ secrets.PROJECT\_ID }}
  - vercel-args: '--prod'

## 6.4 Atividades para implantação da CD

Tabela 7 - Etapas e estimativas de prazos para a implantação da infraestrutura de CD

| Etapa | Descrição                                                         | Prazo estimado |
|-------|-------------------------------------------------------------------|----------------|
| 1     | Conectar repositório GitHub à vercel (já configurado)             | 1h             |
| 2     | Configurar variáveis de ambiente na Vercel (produção/homologação) | 1h             |
| 3     | Criar workflow de CD com vercel-action                            | 2h             |
| 4     | Configurar secrets do GitHub (VERCEL_TOKEN, ORG_ID, PROJECT_ID)   | 1h             |
| 5     | Testar fluxo completo (PR→merge→deploy)                           | 2h             |

Fonte: Elaborado pelo autor (2026)

## 7 MONITORAMENTO E REVISÃO CONTÍNUA

### 7.1 Monitoramento das atividades automatizadas

Para garantir o funcionamento adequado das práticas de CI/CD, serão adotadas as seguintes estratégias de monitoramento:

Tabela 8 - Estratégia de monitoramento, métodos e periodicidade das atividades

| Atividade            | Método de monitoramento                   | Periodicidade          |
|----------------------|-------------------------------------------|------------------------|
| Execução de workflow | GitHub Actions Dashboard (status de runs) | A cada execução        |
| Cobertura de testes  | SonarCloud (relatório automático)         | A cada push na develop |
| Vulnerabilidades     | Snyk, GitHub Security Alerts              | Diário (automático)    |
| Tempo de deploy      | Vercel Analytics                          | A cada deploy          |

|                        |                                         |                  |
|------------------------|-----------------------------------------|------------------|
| Disponibilidade da API | Uptime Robot ou ping simples (cron job) | A cada 5 minutos |
|------------------------|-----------------------------------------|------------------|

Fonte: Elaborado pelo autor (2026)

## 7.2 Política de revisão contínua

Será estabelecida uma rotina de revisão e melhoria do pipeline de CI/CD:

Tabela 9 - Cronograma da política de revisão contínua do pipeline e atribuição de responsabilidade

| Periodicidad<br>e | Atividade                                                                                          | Responsáv<br>el |
|-------------------|----------------------------------------------------------------------------------------------------|-----------------|
| Semanal           | Verificar workflows falhos e corrigir;<br>revisar cobertura de testes                              | Diego           |
| Quinzenal         | Analisar relatório do SonarCloud<br>(dívida técnica, duplicações)                                  | Diego           |
| Mensal            | Atualizar dependências(npm update) e<br>reavaliar npm audit                                        | Diego           |
| Trimestral        | Revisar ferramentas e avaliar novas tecnologias<br>(ex.: substituir GitHub Actions por GitLab CI?) | Diego           |

Fonte: Elaborado pelo autor (2026)

## 7.3 Benefícios do processo proposto

- Feedback imediato: O desenvolvedor sabe em segundos se uma alteração quebrou testes ou violou padrões.
- Redução de erros humanos: Deploys manuais são substituídos por automação confiável.
- Rastreabilidade: Cada commit está associado a uma execução de CI/CD, facilitando auditoria.
- Documentação viva: Os workflows YAML servem como documentação executável do processo.



## 8 RESULTADOS ESPERADOS

Com a implantação do plano de CI/CD, espera-se alcançar os seguintes resultados:

Tabela 10 - Indicadores de desempenho, metas e métricas de sucesso pós-implantação de CI/CD

| Indicador                                   | Situação atual            | Meta após implantação   | Métrica de sucesso     |
|---------------------------------------------|---------------------------|-------------------------|------------------------|
| Tempo de deploy manual                      | ~15 minutos por deploy    | < 2 minutos             | Redução de 85%         |
| Frequência de deploys                       | ~1 por semana             | ~3-5 por semana         | Aumento da agilidade   |
| Defeitos identificados antes da produção    | ~30%                      | ~80%                    | Aumento da qualidade   |
| Cobertura de testes                         | ~40%                      | ≥ 70%                   | SonarCloud             |
| Tempo de feedback sobre qualidade do código | Horas (revisão manual)    | Minutos (CI automático) | Melhoria significativa |
| Vulnerabilidade críticas em produção        | Potencialmente existentes | Zero                    | npm audit passando     |

Fonte: Elaborado pelo autor (2026)

### 8.1 Impacto na extensão comunitária

Além dos benefícios técnicos, a adoção de CI/CD trará impactos positivos para a comunidade:

- **Confiabilidade:** A comunidade poderá confiar que o sistema está sempre funcionando conforme esperado.
- **Interação rápida:** Correções e melhorias solicitadas pelos comerciantes (Lúcia Matos) e moradores (Roseane Soares) poderão ser entregues em dias, não em semanas.
- **Transparência:** O histórico de deploys e mudanças fica registrado, demonstrando o cuidado com a qualidade.

## REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação - Trabalhos acadêmicos - Apresentação. Rio de Janeiro: ABNT, 2011.

GITHUB, Inc. **Sobre a integração contínua com o GitHub Actions** (GitHub Docs). Disponível em: <https://link.ufms.br/rffv>. Acesso em: 22 nov. 2024.

GITHUB, Inc. **Sobre a implantação contínua com o GitHub Actions** (GitHub Docs). Disponível em: <https://link.ufms.br/Yhjvw>. Acesso em: 22 nov. 2024.

HUMBLE, Jez; FARLEY, David. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**. Addison-Wesley, 2010.

MUNIZ, Antonio et al. **Jornada DevOps: unindo cultura ágil, Lean e tecnologia para entregar software com qualidade**. 2. ed. Rio de Janeiro: Brasport, 2020. ISBN 9786599062117.

SONARSOURCE. **SonarCloud Documentation**. Disponível em: <https://sonarcloud.io/documentation>. Acesso em: 25 mai. 2026.

VALENTE, Marco Tulio. **Engenharia de Software moderna: princípios e práticas para desenvolvimento de software com produtividade**. v.1, n.24, 2020. Disponível em: <https://link.ufms.br/bljNp>. Acesso em 20 dez 2024.

VERCEL. **Vercel Documentation - Continuous Deployment**. Disponível em: <https://vercel.com/docs/deployments>. Acesso em: 25 mai. 2026.